



RONINX ACADEMY · DOCUMENTACIÓN TÉCNICA Y FUNCIONAL

ClassVote

Votación de temas en tiempo real para las clases Mondo de cada cinturón.

<https://classvote-lac.vercel.app> · github.com/dojoteamec-ship-it/classvote

Versión del 24 de septiembre de 2026

Contenido

Resumen del proyecto

1 · Reglas de negocio

2 · Manual de uso

3 · Arquitectura

4 · Base de datos

5 · Despliegue y operación

6 · Guía de cambios

7 · Historial

ClassVote · RoninX Academy

Votación de temas en tiempo real para las clases **Mondo** (Q&A) de RoninX Academy. Cada semana, los alumnos de cada cinturón proponen y votan qué tema quieren ver en su clase; el mentor llega a la clase con los temas más votados.

- Cada cinturón (Nivel 1 a 6) tiene **su propia URL y su propio ciclo de votación**. Nunca se mezclan.
- El alumno **no necesita cuenta**: vota desde el enlace de su grupo.
- El mentor tiene **cuenta propia** y un panel para gestionar su cinturón.
- La votación **se cierra sola** a la hora de inicio de cada clase.

Producción: <https://classvote-lac.vercel.app>

Stack

Next.js 16 (App Router) · React 19 · TypeScript · Tailwind CSS v4 · Supabase (Postgres, RLS, Realtime, Auth, pg_cron) · Vercel.

Documentación

#	Documento	Para quién
1	Reglas de negocio : cómo funciona ClassVote y por qué	Todos
2	Manual de uso : alumnos, mentores y administrador	Admin y mentores
3	Arquitectura : stack, carpetas, flujos, seguridad y diseño	Programadores
4	Base de datos : tablas, funciones, permisos y cron	Programadores
5	Despliegue y operación : Vercel, Supabase, GitHub y problemas conocidos	Programadores y admin
6	Guía de cambios : cómo hacer los cambios más comunes	Programadores
7	Historial : qué se construyó y en qué orden	Todos

Arranque rápido (desarrollo)

Requisitos: Node.js 20 o superior y acceso al proyecto de Supabase.

```
cp .env.example .env.local # completar NEXT_PUBLIC_SUPABASE_ANON_KEY (y la service role si  
hace falta)  
npm install  
npm run dev                # http://localhost:3000
```

Chequeos antes de subir cualquier cambio:

```
npm run lint
npm run typecheck
npm run build
```

Next.js 16 tiene cambios importantes respecto a versiones anteriores (por ejemplo, `middleware` ahora se llama `proxy`, y `cookies()` y `params` son asíncronos). Antes de escribir código nuevo, consulta la guía incluida en `node_modules/next/dist/docs/` (ver [AGENTS.md](#)).

URLs principales

Uso	URL
Votación de alumnos	<code>https://classvote-lac.vercel.app/votar/<cinturón></code> (<code>amarillo</code> , <code>naranja</code> , <code>verde</code> , <code>azul</code> , <code>marron</code> , <code>negro</code>)
Registro de mentores	https://classvote-lac.vercel.app/mentor/registro
Entrar (mentores)	https://classvote-lac.vercel.app/mentor/entrar
Panel de un cinturón	<code>https://classvote-lac.vercel.app/mentor/<cinturón></code>
Administración	https://classvote-lac.vercel.app/admin

La lista completa (incluidos GitHub, Vercel y Supabase) está en [Despliegue y operación](#).

1 · Reglas de negocio

Este documento explica **qué hace ClassVote y por qué**. Las decisiones aquí descritas fueron tomadas con el dueño del negocio (Santi, RoninX Academy / Andreti Page LLC). Antes de cambiarlas, confirmarlas con él.

Contexto

RoninX Academy dicta clases en vivo por cinturón. Hay cuatro tipos de sesión:

Tipo	Qué es	¿Entra en ClassVote?
Kata	Masterclass puntual	No (salvo Nivel 6, ver abajo)
Mondo	Preguntas y respuestas; la agenda la deciden los alumnos	Sí
Randori	Práctica guiada	No
Shinsa	Revisión de proyecto	No

ClassVote solo cubre las clases Mondo. Excepción: el Nivel 6 tiene un único espacio semanal tipo Mondo aunque el informe original lo llame "Kata", así que sí entra en votación.

Cinturones y horario

El horario es fijo y está configurado en la base de datos (tabla `cinturones`), en hora de Ecuador (America/Guayaquil, UTC−5, sin horario de verano). No se sincroniza con el calendario de GHL.

Cinturón	Día	Hora	Mentor(es)	Nota
Nivel 0 · Blanco	—	—	—	Fuera de ClassVote (<code>activo = false</code>)
Nivel 1 · Amarillo	Jueves	19:00	Dani (semana A) / Liz-Diana (semana B)	
Nivel 2 · Naranja	Miércoles	18:00	Victor (semana A) / Ismael (semana B)	
Nivel 3 · Verde	Lunes	18:00	Harold / Fer / Sergio	Se abre todas las semanas; el mentor decide Mondo o Randori según el ciclo de 10 semanas
Nivel 4 · Azul	Miércoles	19:00	Efraím	
Nivel 5 · Marrón	Martes	19:00	David + Rafa	El mentor decide Mondo o Randori según lo votado
Nivel 6 · Negro	Domingo	19:00	Andreti	Entra en votación aunque el informe lo llame Kata

Ciclo semanal

Un **ciclo** es la votación de una clase concreta de un cinturón (por ejemplo, "Amarillo, jueves 24 de septiembre").

1. **Apertura automática.** Cuando alguien entra a `/votar/<cinturón>`, o cuando corre el cron cada 5 minutos, se crea el ciclo de la **próxima clase** si todavía no existe. Si hoy es día de clase y la hora ya pasó, la próxima clase es la de la semana siguiente.
2. **Votación.** Mientras el ciclo está `votando`, los alumnos proponen temas y votan.
3. **Cierre automático a la hora de inicio de la clase.** Un cron dentro de Supabase cierra el ciclo a la hora exacta (con hasta 5 minutos de margen). No depende del mentor.
4. **Durante la clase,** el panel del mentor muestra por defecto la clase de hoy (aunque ya haya empezado), con los temas ordenados por votos.

Reglas del mentor sobre el ciclo:

- Puede **cerrar la votación antes** de la hora.
- Puede **reabirla**. Una votación reabierta **ya no se cierra sola**: la cierra el mentor. Así se evita que el cron la vuelva a cerrar minutos después de reabirla.
- Puede marcar el **tipo de sesión**: Mondo (Q&A) o Randori (práctica). Es informativo y útil para los Niveles 3 y 5.
- Puede **ocultar** temas inapropiados o duplicados. Un tema oculto deja de verse y de poder votarse para los alumnos; el mentor lo sigue viendo y puede volver a mostrarlo.

Alumnos

- **No tienen cuenta.** Entran desde el enlace de su cinturón (que se publicará en el grupo de GHL correspondiente).
- **Pueden votar varios temas, una vez cada uno.** El voto no se puede deshacer.
- La identificación es **anónima por navegador** (un identificador aleatorio guardado en el navegador). Riesgo aceptado para v1: en modo incógnito o desde otro dispositivo se puede volver a votar. Para una votación de temas de clase se consideró aceptable.
- Al proponer un tema, el **nombre es opcional**; si no lo ponen aparece como "Anónimo".
- El tema debe tener entre 3 y 200 caracteres; el nombre, hasta 40.

Mentores y administrador

- **Cada mentor crea su propia cuenta** (correo y contraseña) en `/mentor/registro`. Nunca hay cuentas compartidas ni acceso por SSO de GHL.
- Toda cuenta nueva queda **pendiente**: no ve nada hasta que el **administrador** la aprueba y le asigna uno o más cinturones.
- Un mentor solo puede gestionar **los cinturones que tiene asignados**.

- El **administrador** (rol `admin`, "super admin") ve y gestiona **todos** los cinturones y además puede: aprobar o rechazar cuentas, desactivar o reactivar mentores, asignar o quitar cinturones, dar o quitar el rol de admin y restablecer contraseñas.
- Un admin no puede cambiarse a sí mismo el estado ni el rol, para no quedarse sin acceso por error.
- Si un mentor olvida su contraseña, el admin le genera una **contraseña temporal** desde Administración y se la envía por un canal privado; el mentor la cambia en "Mi cuenta". No hay recuperación por correo (ver [problemas conocidos](#)).

Riesgos aceptados para la versión 1

Riesgo	Por qué se aceptó
Un voto por tema por navegador , no por persona	Los alumnos no tienen cuenta; es una votación de temas, no una elección
No hay límite de temas por alumno	El mentor puede ocultar spam o duplicados
Un tema oculto sigue visible para quien ya tenía la página abierta, hasta recargar (no puede votarlo)	La base de datos ya bloquea el voto; es solo visual
Sin recuperación de contraseña por correo	El servidor de correo gratuito de Supabase no envía a correos externos; el admin restablece manualmente
Horario fijo, no sincronizado con GHL	Suficiente para v1; cambiar el horario es una línea de SQL

Pendiente (fase 5)

Publicar cada URL `/votar/<cinturón>` en la pestaña Learning/Events del grupo de GHL correspondiente, **después** de que el equipo revise y apruebe el diseño. Si se incrusta en un iframe, verificar que el voto funcione en Safari/iPhone (el almacenamiento del navegador puede estar bloqueado dentro de iframes de terceros).

2 · Manual de uso

Guía paso a paso, sin lenguaje técnico, para el administrador, los mentores y los alumnos.

Para alumnos

1. Entra desde el enlace de tu cinturón (lo encontrarás en tu grupo). Ejemplo:
`https://classvote-lac.vercel.app/votar/amarillo`.
2. Verás la fecha de la próxima clase Mondo y una **cuenta regresiva** hasta el cierre.
3. **Proponer un tema:** escríbelo en "Propón un tema" y, si quieres, tu nombre. Pulsa **Proponer tema**.
4. **Votar:** pulsa **Votar** en todos los temas que te interesen. Cada tema admite un solo voto tuyo y el voto no se puede deshacer. Verás el sello  en los que ya votaste.
5. Los conteos se actualizan **en vivo**, sin recargar.
6. Cuando empieza la clase, la votación se cierra y se muestran los temas elegidos. La siguiente vez que entres ya estará abierta la votación de la semana siguiente.

Para mentores

Crear tu cuenta (una sola vez)

1. Entra a <https://classvote-lac.vercel.app/mentor/registro>.
2. Escribe tu nombre, tu correo y una contraseña de mínimo 8 caracteres.
3. Tu cuenta queda **pendiente**. Avísale al administrador para que la apruebe y te asigne tu cinturón. Mientras tanto verás el mensaje "Tu cuenta está pendiente".

Entrar

- <https://classvote-lac.vercel.app/mentor/entrar> con tu correo y contraseña.
- En **Cinturones** verás las tarjetas de los cinturones que tienes asignados. Haz clic en uno para abrir su panel.

Panel del cinturón

Qué ves o haces	Cómo
Enlace para alumnos	Arriba a la derecha. Copiar enlace lo copia listo para pegar en el grupo.
Cifras	Temas, votos, tiempo hasta el cierre ("Cierra en") y estado en vivo.
Clase	Por defecto, la de hoy (aunque ya haya empezado); si no hay clase hoy, la próxima. Las fechas de arriba permiten ver clases anteriores.

Qué ves o haces	Cómo
Tipo de sesión	Elige Mundo (Q&A) o Randori (práctica) . Queda guardado al instante.
Cerrar votación ahora	Cierra antes de la hora (pide confirmación).
Reabrir votación	Vuelve a abrirla. Ojo: una votación reabierta ya no se cierra sola ; ciérrala tú al terminar.
Ocultar / Mostrar	Junto a cada tema. Un tema oculto desaparece para los alumnos y no se puede votar; tú lo sigues viendo tachado.

Todo se actualiza en vivo: durante la clase puedes dejar el panel abierto y ver cómo llegan los votos.

Mi cuenta

Haz clic en tu nombre (arriba a la derecha) para ir a **Mi cuenta** y cambiar tu contraseña. Si el administrador te dio una contraseña temporal, cámbiala aquí.

Para el administrador

Entra a <https://classvote-lac.vercel.app/admin> (o el enlace **Administrar** de la barra superior). Arriba verás cuántos mentores hay pendientes, activos y cuántos admins.

Acción	Cómo
Aprobar un mentor nuevo	En su tarjeta (resaltada), pulsa Aprobar . Para rechazarlo, Rechazar .
Asignar cinturones	En la tarjeta de un mentor activo, haz clic en los cinturones (se encienden al asignarse). Clic de nuevo para quitarlo. Un mentor puede tener varios.
Desactivar / Reactivar	Cuando un mentor deja de dar clases, Desactivar . Pierde el acceso sin borrar su historial.
Hacer admin / Quitar admin	Da o quita acceso total. Los admins ven todos los cinturones, por eso no muestran asignaciones.
Restablecer contraseña	Genera una contraseña temporal que se muestra una sola vez. Cópiala y envíasela al mentor por un canal privado; pídele que la cambie en Mi cuenta. También confirma su correo si estaba pendiente.

No puedes cambiar tu propio estado ni tu propio rol (para no quedarte afuera por error).

Alta de un mentor nuevo (paso a paso)

1. Envíale el enlace **/mentor/registro**.
2. Cuando se registre, entra a **Administración**: aparecerá como **Pendiente**.
3. **Aprobar** y asignarle su(s) **cinturón(es)**.

4. Listo: al recargar, verá su panel.

Cuando cambia el mentor de un cinturón

1. Asigna el cinturón al mentor nuevo.
2. Quita el cinturón al anterior o **desactívalo** si ya no dará clases.

Preguntas frecuentes

Un alumno dice que no puede votar. Revisa que la votación esté abierta en el panel. Si ya votó ese tema desde ese navegador, verá "Votaste". Si el tema fue ocultado, no puede votarse.

Quiero volver a abrir una votación que se cerró sola. Pulsa **Reabrir votación** en el panel y ciérrala manualmente al terminar.

Un mentor no puede entrar. Revisa en Administración que esté **Activo**. Si olvidó la contraseña, usa **Restablecer contraseña**.

Aparecen temas de prueba. Ocúltalos desde el panel del cinturón.

3 · Arquitectura

Visión general

```
Navegador (alumno / mentor)
| HTML renderizado en servidor + componentes de cliente
▼
Vercel · Next.js 16 (App Router)
| Server Components y Server Actions con la sesión del usuario (cookies)
| proxy.ts refresca la sesión en /mentor y /admin
▼
Supabase
├─ Postgres + RLS ← toda la autorización real vive aquí
├─ Auth           ← cuentas de mentores (correo + contraseña)
├─ Realtime       ← cambios de temas y ciclos en vivo al navegador
└─ pg_cron        ← cierra ciclos vencidos cada 5 minutos
```

Principio central: la base de datos es la que decide quién puede hacer qué (Row Level Security + permisos por columna). La app nunca usa la llave de servicio para operaciones normales: alumnos y mentores escriben con su propio rol (`anon` o `authenticated`), así que aunque alguien manipule el navegador, Postgres rechaza lo que no corresponde. La llave de servicio solo se usa para restablecer contraseñas (API de administración de Auth), siempre después de comprobar que el usuario es admin.

Stack

Pieza	Versión	Notas
Next.js	16.3.6	App Router, Turbopack. <code>middleware</code> → <code>proxy.ts</code> ; <code>cookies()</code> , <code>params</code> y <code>searchParams</code> son Promise
React	19.2	<code>useActionState</code> , <code>useSyncExternalStore</code> ; reglas estrictas de lint (no <code>setState</code> síncrono en efectos)
TypeScript	5	<code>npm run typecheck = next typegen && tsc --noEmit</code> (genera <code>PageProps</code> / <code>LayoutProps</code>)
Tailwind CSS	4	Configuración en CSS (<code>@theme</code> , <code>@utility</code>) dentro de <code>app/globals.css</code> ; no hay <code>tailwind.config</code>
@supabase/ssr	0.12	Clientes de navegador y servidor con cookies
@supabase/supabase-js	2.117	Cliente de administración (service role)
Supabase	—	Postgres 15+, RLS, Realtime, Auth, <code>pg_cron</code>
Vercel	Hobby	Despliegue automático desde <code>main</code> (producción) y ramas (preview)

Estructura de carpetas

app/	
layout.tsx	Layout raíz: fuentes (Inter), fondo decorativo, metadatos
globals.css	Sistema de diseño: tokens de color, utilidades (tarjeta, botones, campo...), animaciones
page.tsx	Portada
icon.png, apple-icon.png, favicon.ico	Íconos (convención de archivos de Next)
votar/[slug]/	
page.tsx	Servidor: carga cinturón, abre/lee ciclo (rpc ciclo_actual) y temas
votacion.tsx	Cliente: proponer, votar, tiempo real, cuenta regresiva
mentor/	
acciones.ts	Server Actions: entrar, registrarse, salir, cambiar contraseña,
acciones del panel	
registro/, entrar/	Pantallas de acceso (usan pantalla-acceso.tsx + formulario-
cuenta.tsx)	
page.tsx	Inicio del mentor: sus cinturones (admin: todos)
cuenta/page.tsx	Mi cuenta (cambiar contraseña)
[slug]/page.tsx	Servidor: panel del cinturón, selección de clase
[slug]/panel-ciclo.tsx	Cliente: cifras, tipo de sesión, cerrar/reabrir, ocultar temas,
tiempo real	
[slug]/copiar-enlace.tsx	
encabezado.tsx, contenedor.tsx	Barra superior y marco de páginas con sesión
admin/	
page.tsx	Gestión de mentores
acciones.ts	Server Actions de admin (estado, rol, cinturones, restablecer
contraseña)	
boton-restablecer.tsx	Cliente: muestra la contraseña temporal
components/	UI compartida: marca (logo/emblema), fondo, obi (cinturón), rótulos,
	piezas de la lista de temas, cuenta regresiva
lib/	
auth.ts	obtenerSesion / requerirMentor / requerirMentorActivo / requerirAdmin
supabase/client.ts	Cliente de navegador (anon key)
supabase/server.ts	Cliente de servidor con cookies (server-only)
supabase/proxy.ts	Refresco de sesión para proxy.ts
supabase/admin.ts	Cliente con service role (server-only; solo Production tiene la
llave)	
supabase/env.ts	Lectura validada de variables públicas
use-ciclo-en-vivo.ts	Hook de Realtime compartido por alumno y mentor
fecha.ts	Fechas en hora de Ecuador (formatearClase, hoyEnEcuador,
instanteClase)	
cinturones.ts	Color y kanji de cada cinturón (obi)
temas.ts	Columnas a pedir y etiquetas de tipo de sesión
fingerprint.ts	Identificador anónimo del navegador
types/database.ts	Tipos de las tablas (escritos a mano)
migrations/	SQL versionado (0001 → 0004); se aplica a mano en Supabase
seed.sql	Los 7 cinturones con su horario
public/marca/	Logotipo RONIN y emblema del ronin (PNG transparentes)
proxy.ts	Refresca la sesión en /mentor/* y /admin/*

Rutas

Ruta	Tipo	Acceso	Qué hace
/	Estática	Público	Portada

Ruta	Tipo	Acceso	Qué hace
/votar/[slug]	Dinámica	Público	Votación del cinturón. blanco o un slug inexistente dan 404
/mentor/registro	Dinámica	Sin sesión	Crear cuenta (queda pendiente)
/mentor/entrar	Dinámica	Sin sesión	Iniciar sesión
/mentor	Dinámica	Con sesión	Cinturones del mentor, o aviso de pendiente/desactivado
/mentor/cuenta	Dinámica	Con sesión	Cambiar contraseña
/mentor/[slug]	Dinámica	Mentor activo del cinturón o admin	Panel del cinturón (?ciclo=<id> para otra clase)
/admin	Dinámica	Admin activo	Gestión de mentores

Las páginas protegidas llaman a `requerirMentor()`, `requerirMentorActivo()` o `requerirAdmin()` (en `lib/auth.ts`), que redirigen si no corresponde. Aun así, **la autorización definitiva la hace RLS**: si una página olvidara el chequeo, la base igual no devolvería ni modificaría datos ajenos.

Flujos principales

Alumno vota

1. `app/votar/[slug]/page.tsx` (servidor) lee el cinturón (RLS solo muestra los activos), llama a `rpc('ciclo_actual', { p_slug })`, que crea el ciclo de la próxima clase si no existe y lo devuelve, y carga los temas visibles.
2. `votacion.tsx` (cliente) obtiene el identificador anónimo (`lib/fingerprint.ts`, guardado en `localStorage`) y los temas ya votados en este navegador.
3. **Votar** inserta en `votos` con la clave pública. Un trigger suma 1 a `temas.votos_count`. El voto duplicado lo rechaza la restricción `unique (tema_id, alumno_fingerprint)` (código 23505).
4. **Proponer** inserta en `temas` (solo puede escribir `ciclo_id`, `texto`, `alumno_alias`).
5. **Tiempo real**: `useCicloEnVivo` se suscribe a `temas` (filtrado por ciclo) y a `ciclos_semanales` (ese ciclo). Los votos llegan como `UPDATE` de `temas.votos_count`, así que el navegador **nunca lee la tabla** `votos` y los identificadores no se exponen.
6. Las acciones son **optimistas**: se reflejan al instante y se revierten si la base las rechaza.

Mentor se registra y entra

1. `registrarse` (Server Action) llama a `supabase.auth.signUp` con `nombre` en los metadatos.

2. Un trigger en `auth.users` crea la fila en `mentores` con `estado = 'pendiente'`, `rol = 'mentor'`.
3. El admin aprueba (`estado = 'activo'`) y asigna cinturones (`mentor_cinturones`).
4. `entrar` llama a `signInWithPassword`; la sesión queda en cookies y `proxy.ts` la refresca en cada navegación de `/mentor` y `/admin`.

Mentor gestiona su cinturón

`panel-ciclo.tsx` llama a Server Actions (`cambiarTipoSesion`, `cambiarEstadoCiclo`, `cambiarOculto`) que hacen `update ... select('id')` con la sesión del mentor. Si RLS no lo permite, no se actualiza ninguna fila y se devuelve error; el cambio optimista se revierte.

Cierre automático

`pg_cron` ejecuta `cerrar_ciclos_vencidos()` cada 5 minutos: cierra los ciclos `votando` cuya hora de clase ya pasó (salvo los `reabierto`) y crea el ciclo de la próxima clase de cada cinturón. Ver [Base de datos](#).

Seguridad

Capa	Qué protege
RLS en todas las tablas	Lectura pública solo de lo necesario; escritura según rol y estado del ciclo
Permisos por columna	<code>anon</code> solo escribe <code>texto / alumno_alias</code> (temas) y <code>tema_id / alumno_fingerprint</code> (votos); no puede inflar <code>votos_count</code> , editar ni borrar. Mentores solo actualizan <code>estado</code> , <code>tipo_sesion</code> , <code>cerrado_en</code> , <code>reabierto</code> (ciclos) y <code>oculto</code> (temas)
Funciones security definer	<code>es_admin()</code> , <code>puede_gestionar()</code> , <code>ciclo_actual()</code> , triggers y cron, con <code>search_path</code> fijo
Mentor pendiente	No puede hacer nada hasta ser aprobado; tampoco puede aprobarse a sí mismo
Service role	Solo en el servidor (<code>server-only</code>), solo en Production, solo para restablecer contraseñas tras <code>requerirAdmin()</code>
Anon key	Pública por diseño (va al navegador); la seguridad la da RLS

Todo esto está probado en un Postgres local con los roles de Supabase simulados (ver historial de los PR).

Sistema de diseño

Todo el diseño está en `app/globals.css` y `components/`.

- **Identidad:** RONIN "ronin cibernético". Fondo negro azulado, **un solo acento cian eléctrico**, rojo solo para errores. Estructura inspirada en Apple: tipografía grande con

tracking cerrado, mucho aire, vidrio sobrio y botones tipo píldora.

- **Tokens** (@theme): `noche-*` (fondos), `cian-*` (acento), `washi` (texto), `matcha` (en vivo), `shu-*` (errores), `ai-400` (barras).
- **Utilidades** (@utility): `tarjeta`, `tarjeta-interactiva`, `boton-primario`, `boton-secundario`, `campo`, `rotulo`, `titular`, `texto-acento`.
- **Animaciones**: `aparecer`, `latido` (contador), `sello` y `onda` (al votar), `pulso` (en vivo), `niebla`, `flotar`. Todo se desactiva con `prefers-reduced-motion`.
- **Componentes**: `Marca / Emblema` (logo real en `public/marca/`), `FondoDojo` (ronin frente a la luna, halo cian, circuito, niebla, grano), `Obi` (cinturón con su color), `Rotulo`, `EstadoVotacion`, `Puesto / BarraVotos / Contador`, `CuentaRegresiva`.
- **Fuentes**: Inter desde Google Fonts vía `<link>` en el layout. No se usa `next/font` porque falla al compilar en entornos sin red.
- **Íconos**: generados desde el emblema del logo (`app/icon.png`, `app/apple-icon.png`, `app/favicon.ico`).

Convenciones de código

- Nombres de dominio y comentarios **en español** (`cinturon`, `ciclo`, `tema`, `votar...`).
- Comentarios solo donde el *por qué* no es obvio.
- Componentes de servidor por defecto; `"use client"` solo donde hay interacción o tiempo real.
- Funciones que usan servidor y cliente van en `lib/` **sin** `"use client"` (una función exportada desde un archivo de cliente no puede llamarse desde el servidor).
- Estado del navegador (`localStorage`) se lee con `useSyncExternalStore` para no descuadrar la hidratación.

4 · Base de datos

Supabase (Postgres). Proyecto `cvpvkactwtymvckbpxhm`. El esquema vive en `migrations/` y se aplica **a mano** en el SQL Editor de Supabase, en orden. Todas las migraciones desde la 0002 son **idempotentes** (se pueden correr más de una vez).

Migración	Qué agrega
<code>0001_init.sql</code>	Tablas base, enums, RLS inicial
<code>seed.sql</code>	Los 7 cinturones con su horario
<code>0002_fase2_votacion.sql</code>	<code>votos_count</code> + trigger, permisos por columna del alumno, <code>ciclo_actual()</code> , Realtime
<code>0003_fase3_mentores.sql</code>	Estado y rol del mentor, registro automático, <code>es_admin()</code> , <code>puede_gestionar()</code> , temas ocultos, RLS del panel
<code>0004_fase4_cierre_automatico.sql</code>	<code>reabierto</code> , <code>cerrar_ciclos_vencidos()</code> , <code>pg_cron</code> cada 5 min

Modelo

```
graph TD
    cinturones -- 1 --> * ciclos_semanales
    cinturones -- 1 --> * temas
    cinturones -- 1 --> * votos
    cinturones -- * --> mentor_cinturones
    mentor_cinturones -- * --> mentores
    mentores -- 1 --> auth.users
```

cinturones

Columna	Tipo	Notas
<code>id</code>	text PK	<code>nivel_0</code> ... <code>nivel_6</code>
<code>nombre</code>	text	"Amarillo — Nivel 1"
<code>orden</code>	smallint	Orden de presentación
<code>slug</code>	text único	Se usa en las URL: <code>amarillo</code> , <code>naranja</code> , <code>verde</code> , <code>azul</code> , <code>marron</code> , <code>negro</code> , <code>blanco</code>
<code>dia_semana</code>	smallint 0-6	0 = domingo ... 6 = sábado
<code>hora_local</code>	time	Hora de la clase en America/Guayaquil
<code>activo</code>	boolean	<code>false</code> = fuera de ClassVote (Nivel 0)

ciclos_semanales

Columna	Tipo	Notas
id	uuid PK	
cinturon_id	text FK	
fecha_clase	date	Única junto con <code>cinturon_id</code>
estado	enum <code>ciclo_estado</code>	<code>votando</code> · <code>cerrado</code>
tipo_sesion	enum <code>sesion_tipo</code>	<code>qa</code> · <code>practica</code> · null (sin definir)
cerrado_en	timestampz	Momento del cierre
reabierto	boolean	<code>true</code> si un mentor lo reabrió; el cron ya no lo cierra

temas

Columna	Tipo	Notas
id	uuid PK	
ciclo_id	uuid FK (cascade)	
texto	text	3-200 caracteres
alumno_alias	text	Opcional, 1-40 caracteres
votos_count	integer	Mantenido por trigger; nadie lo escribe directamente
oculto	boolean	Oculto por un mentor
creado_en	timestampz	Desempate del ranking

votos

Columna	Tipo	Notas
id	uuid PK	
tema_id	uuid FK (cascade)	
alumno_fingerprint	text	Identificador anónimo del navegador (16-64 caracteres)
creado_en	timestampz	

Restricción `unique (tema_id, alumno_fingerprint)` : un voto por tema por navegador.

mentores

Columna	Tipo	Notas
id	uuid PK	

Columna	Tipo	Notas
nombre	text	Del registro
email	text único	En minúsculas
auth_user_id	uuid FK → auth.users	
estado	enum mentor_estado	pendiente · activo · inactivo
rol	enum mentor_rol	mentor · admin
creado_en	timestampz	

mentor_cinturones

Asignación mentor ↔ cinturón (PK compuesta mentor_id, cinturon_id).

Funciones

Función	Tipo	Quién la ejecuta	Qué hace
ciclo_actual(p_slug)	security definer, setof ciclos_semanales	anon , authenticated	Calcula la próxima clase del cinturón (hora de Ecuador), crea el ciclo si no existe y lo devuelve. Nunca lo cierra
es_admin()	security definer, boolean	todos	¿El usuario actual es admin activo?
puede_gestionar(p_cinturon_id)	security definer, boolean	todos	¿Es admin, o mentor activo asignado a ese cinturón?
sync_votos_count()	trigger en votos	—	Suma o resta en temas.votos_count
crear_mentor_desde_auth()	trigger en auth.users	—	Crea la fila en mentores como pendiente al registrarse
cerrar_ciclos_vencidos()	security definer, integer	solo postgres (cron)	Cierra ciclos vencidos y abre la próxima clase de cada cinturón

es_admin() y **puede_gestionar()** concentran toda la lógica de permisos. Para agregar un rol nuevo (por ejemplo "coordinador"), se agrega el valor al enum mentor_rol y se ajustan esas dos funciones; las políticas no cambian.

Permisos (RLS y columnas)

Tabla	Leer	Insertar	Actualizar	Borrar
cinturones	Todos, solo activo	—	—	—

Tabla	Leer	Insertar	Actualizar	Borrar
ciclos_semanales	Todos	Solo vía <code>ciclo_actual()</code> y <code>cron</code>	<code>puede_gestionar(cinturon)</code> ; — columnas <code>estado</code> , <code>tipo_sesion</code> , <code>cerrado_en</code> , <code>reabierto</code>	—
temas	Visibles para todos; ocultos solo si <code>puede_gestionar</code>	Todos, si el ciclo está <code>votando</code> ; columnas <code>ciclo_id</code> , <code>texto</code> , <code>alumno_alias</code>	<code>puede_gestionar</code> ; solo <code>oculto</code>	—
votos	Nadie (el conteo está en <code>temas</code>)	Todos, si el ciclo está <code>votando</code> y el tema no está oculto; columnas <code>tema_id</code> , <code>alumno_fingerprint</code>	—	—
mentores	La fila propia, o todo si <code>es_admin()</code>	Solo el trigger de registro	<code>es_admin()</code> ; columnas <code>nombre</code> , <code>estado</code> , <code>rol</code>	—
mentor_cinturon	Las propias, o todo si <code>es_admin()</code>	<code>es_admin()</code>	—	<code>es_admin()</code>

Tiempo real

Publicación `supabase-realtime` con las tablas `temas` y `ciclos_semanales`. Realtime respeta RLS: un alumno no recibe eventos de temas ocultos, y un mentor sí.

Cierre automático (pg_cron)

```
-- Programado por la migración 0004
select cron.schedule('classvote-cerrar-ciclos', '* / 5 * * * *',
  $$select public.cerrar_ciclos_vencidos()$$);
```

Revisar que esté activo y sus últimas ejecuciones:

```
select jobname, schedule, active from cron.job;
select status, return_message, start_time
from cron.job_run_details order by start_time desc limit 10;
```

`return_message` indica cuántas filas se cerraron (1 row = la función corrió). Para pausarlo: `select cron.unschedule('classvote-cerrar-ciclos');` y para reactivarlo, volver a correr la sección 3 de la migración 0004.

Consultas útiles

```
-- Temas de la próxima clase de un cinturón, por votos
select t.texto, t.alumno_alias, t.votos_count, t.oculto
from temas t join ciclos_semanales c on c.id = t.ciclo_id
where c.id = (select id from ciclo_actual('amarillo'))
order by t.votos_count desc, t.creado_en;

-- Mentores y sus cinturones
select m.nombre, m.email, m.estado, m.rol, string_agg(mc.cinturon_id, ', ') as cinturones
from mentores m left join mentor_cinturones mc on mc.mentor_id = m.id
group by m.id order by m.estado, m.nombre;

-- Hacer admin a alguien (primer admin, o si se pierde el acceso)
update mentores set estado = 'activo', rol = 'admin' where email = 'correo@ejemplo.com';

-- Confirmar a mano el correo de un usuario
update auth.users set email_confirmed_at = now() where email = 'correo@ejemplo.com';

-- Cambiar el horario de un cinturón
update cinturones set dia_semana = 4, hora_local = '20:00' where slug = 'amarillo';
```

Tipos en el código

`types/database.ts` tiene los tipos escritos a mano. Si cambias el esquema, actualízalos. A futuro se pueden generar con `supabase gen types typescript --linked > types/database.ts` una vez que el CLI de Supabase esté enlazado al proyecto.

5 · Despliegue y operación

Servicios

Servicio	Cuenta / proyecto	Para qué
GitHub	dojoteamec-ship-it/classvote (público)	Código
Vercel	Team dojoteamec-ship-its-projects (plan Hobby), proyecto classvote	Hosting y despliegues
Supabase	Proyecto cvpvkactwtymvckbpxhm	Base de datos, Auth, Realtime, cron

URLs

App

Uso	URL
Producción	https://classvote-lac.vercel.app
Votación Amarillo · Nivel 1	https://classvote-lac.vercel.app/votar/amarillo
Votación Naranja · Nivel 2	https://classvote-lac.vercel.app/votar/naranja
Votación Verde · Nivel 3	https://classvote-lac.vercel.app/votar/verde
Votación Azul · Nivel 4	https://classvote-lac.vercel.app/votar/azul
Votación Marrón · Nivel 5	https://classvote-lac.vercel.app/votar/marron
Votación Negro · Nivel 6	https://classvote-lac.vercel.app/votar/negro
Registro de mentores	https://classvote-lac.vercel.app/mentor/registro
Entrar	https://classvote-lac.vercel.app/mentor/entrar
Panel de un cinturón	<a href="https://classvote-lac.vercel.app/mentor/<cinturón>">https://classvote-lac.vercel.app/mentor/<cinturón>
Mi cuenta	https://classvote-lac.vercel.app/mentor/cuenta
Administración	https://classvote-lac.vercel.app/admin

Gestión

Uso	URL
Repositorio	https://github.com/dojoteamec-ship-it/classvote
Pull Requests	https://github.com/dojoteamec-ship-it/classvote/pulls
Proyecto en Vercel	https://vercel.com/dojoteamec-ship-its-projects/classvote

Uso	URL
Variables de entorno (Vercel)	https://vercel.com/dojoteamec-ship-its-projects/classvote/settings/environment-variables
Proyecto en Supabase	https://supabase.com/dashboard/project/cvpvkactwtymvckbpxhm
SQL Editor	https://supabase.com/dashboard/project/cvpvkactwtymvckbpxhm/sql/new
Usuarios (Auth)	https://supabase.com/dashboard/project/cvpvkactwtymvckbpxhm/auth/users
Proveedores de acceso	https://supabase.com/dashboard/project/cvpvkactwtymvckbpxhm/auth/providers

Variables de entorno

Configuradas en Vercel. **Nunca** se suben al repositorio (`.env*` está en `.gitignore` ; `.env.example` es la plantilla sin valores).

Variable	Tipo en Vercel	Entornos	Uso
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Config	Production, Preview, Development	URL del proyecto de Supabase
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Config	Production, Preview, Development	Llave pública (va al navegador; la seguridad la da RLS)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Secret	Solo Production	Solo para restablecer contraseñas desde /admin. Salta RLS: nunca exponerla

Las variables con prefijo `NEXT_PUBLIC_` deben ser tipo **Config** (Vercel no permite que sean "Secret" porque llegan al navegador). Al cambiar una variable hay que hacer **Redeploy** para que el build la tome.

Si una llave se expone (por ejemplo, pegada en un chat), regenerarla en Supabase → Project Settings → API Keys y actualizarla en Vercel.

Configuración de Supabase Auth

En **Authentication** → **Sign In / Providers**:

- **Allow new users to sign up:** encendido (los mentores se registran solos).
- **Confirm email: apagado.** La aprobación del admin reemplaza la confirmación, y el servidor de correo gratuito de Supabase no envía a correos externos.
- **Email provider** → **Enable email provider: encendido** (si se apaga, nadie puede registrarse ni entrar).
- **Minimum password length:** 8 (coincide con la app).

En **Authentication** → **URL Configuration**, **Site URL** debe ser `https://classvote-lac.vercel.app`.

Cómo se publica un cambio

```
rama de trabajo —push→ Preview en Vercel (URL propia, misma base de datos)
|
└─ Pull Request a main —Merge→ Producción (classvote-lac.vercel.app)
```

1. Trabajar en una rama; cada push genera un **Preview** en Vercel (sirve para revisar diseño y funcionalidad antes de publicar).
2. Correr `npm run lint`, `npm run typecheck` y `npm run build` antes de subir.
3. Abrir un Pull Request hacia `main`. Si el cambio incluye una migración SQL, **correrla en Supabase antes del merge** (el PR debe decirlo en "Antes de hacer Merge").
4. Hacer **Merge con la cuenta** `dojoteamec-ship-it` (ver problemas conocidos).
5. Vercel despliega `main` a producción en 1-2 minutos.

Importante: los Preview usan **la misma base de datos** que producción. Lo que se vote o proponga en un Preview aparece en producción; usar datos de prueba con cuidado y ocultarlos después.

Aplicar una migración

1. Abrir el archivo en GitHub → **Copy raw file**.
2. Supabase → **SQL Editor** → New query → pegar → **Run**.
3. Verificar con las consultas del final de la migración o de [Base de datos](#).

Problemas conocidos (y cómo se resolvieron)

Síntoma	Causa	Solución
Deployment "Blocked" en Vercel ("the commit author did not have contributing access")	Plan Hobby: con repo privado , Vercel solo despliega commits cuyo autor es el dueño de la cuenta	El repo es público . Hacer los merges con <code>dojoteamec-ship-it</code> . Si aun así se bloquea, Promote un Preview ya listo del mismo código
El repo no aparece para conectar en Vercel	El repo estaba en otra cuenta de GitHub (SantiJimenezIA)	Se transfirió a <code>dojoteamec-ship-it</code>
"No se pudo crear la cuenta" al registrarse	Proveedor Email apagado en Supabase	Encender Enable email provider
"Revisa tu correo para confirmar la cuenta" y luego no puede entrar	Confirm email encendido	Apagarlo. Para cuentas ya creadas: <code>update auth.users set email_confirmed_at = now() where email = '...';</code> o Restablecer contraseña desde /admin (también confirma el correo)

Síntoma	Causa	Solución
"Restablecer contraseña" dice "No disponible en este entorno"	Falta <code>SUPABASE_SERVICE_ROLE_KEY</code> (solo existe en Production)	Usarlo en producción, o agregar la variable al entorno
La página del alumno dice "La votación no está disponible"	Falta una migración o Supabase no responde	Verificar migraciones 0002-0004 y el estado de Supabase
Una votación reabierta no se cierra sola	Comportamiento esperado (<code>reabierto = true</code>)	La cierra el mentor desde el panel
El favicon viejo sigue apareciendo	Caché del navegador	Recarga forzada (Cmd/Ctrl + Shift + R)
El build falla con <code>next/font/google</code> en entornos sin red	<code>next/font</code> descarga fuentes al compilar	No usar <code>next/font</code> ; las fuentes se cargan con <code><link></code> en <code>app/layout.tsx</code>

Monitoreo básico

- **Vercel → Deployments / Logs:** errores de servidor (las Server Actions registran el código de error de Supabase con `console.error`).
- **Supabase → Logs → Auth:** errores de registro y login.
- **Cron:** `select ... from cron.job_run_details ...` (ver [Base de datos](#)).

Respaldo

Supabase hace respaldos diarios en su plan. El esquema completo se puede reconstruir desde cero con `migrations/0001` → `0004` + `seed.sql` , en ese orden.

6 · Guía de cambios

Recetas para los cambios más probables. Antes de cualquier cambio:

1. Leer [Arquitectura](#) y, si toca datos, [Base de datos](#).
2. Next.js 16 difiere de versiones anteriores: consultar `node_modules/next/dist/docs/`.
3. Al terminar: `npm run lint && npm run typecheck && npm run build`, y publicar por Pull Request ([Despliegue](#)).

Datos y horarios (sin tocar código)

Cambiar el día u hora de clase de un cinturón

```
update cinturones set dia_semana = 4, hora_local = '20:00' where slug = 'amarillo';
```

`dia_semana` : 0 = domingo, 1 = lunes ... 6 = sábado. Aplica desde el **próximo** ciclo que se cree; si ya existe el de esta semana con la fecha anterior, se puede borrar (si no tiene temas) o cerrar a mano.

Activar o desactivar un cinturón

```
update cinturones set activo = false where slug = 'verde';
```

Un cinturón inactivo da 404 en `/votar/<slug>` y el cron no le abre ciclos. Para activar el Nivel 0 (Blanco) hay que darle también `dia_semana` y `hora_local`.

Reglas de votación

Limitar a N votos por alumno por ciclo

Hoy un alumno puede votar todos los temas (una vez cada uno). Para limitar:

1. **Base de datos (obligatorio):** agregar a la política `"votos: insertar mientras vota"` una condición que cuente los votos del mismo `alumno_fingerprint` en el ciclo, por ejemplo con una función `security definer votos_en_ciclo(fingerprint, ciclo_id)`.
2. **Interfaz:** en `app/votar/[slug]/votacion.tsx`, deshabilitar **Votar** al llegar al límite y mostrar cuántos votos quedan. Manejar el error de RLS (42501).

Permitir quitar un voto

Requiere una política `delete` en `votos` para el mismo fingerprint (el trigger `sync_votos_count` ya resta en `DELETE`) y un botón en `votacion.tsx`. Nota: el fingerprint lo envía el navegador, así que cualquiera que conozca un fingerprint podría borrar ese voto; evaluar el riesgo.

Cambiar la hora de cierre (por ejemplo, 1 hora antes de la clase)

En `migrations/`, crear `0005_...sql` que redefina `cerrar_ciclos_vencidos()` restando un intervalo:

```
and (cs.fecha_clase + ci.hora_local) at time zone 'America/Guayaquil' - interval '1 hour' <=
now()
```

Ajustar también `ciclo_actual()` (cuándo pasa a la semana siguiente) y la cuenta regresiva (`instanteClase` en `lib/fecha.ts`) para que muestren la misma hora.

Roles y permisos

Agregar un rol (por ejemplo, "coordinador" que ve todos los cinturones sin gestionar mentores)

1. Migración: `alter type mentor_rol add value 'coordinador';`
2. Ajustar `puede_gestionar()` para que el coordinador activo devuelva `true`.
3. `types/database.ts`: agregar el valor a `MentorRol`.
4. `app/mentor/page.tsx`: que el coordinador vea todos los cinturones (hoy solo `admin`).
5. `app/admin/page.tsx`: botón para asignar el rol.

La autorización real queda en la base; la interfaz solo decide qué mostrar.

Interfaz y diseño

Cambiar colores

Todos los colores están en `@theme` dentro de `app/globals.css`. El acento es la familia `cian-*`; cambiar esos valores actualiza toda la app. Los colores de cada cinturón (obi) están en `lib/cinturones.ts`.

Cambiar el logo o el emblema

Reemplazar `public/marca/ronin-logo.png` (logotipo, PNG blanco transparente) y `public/marca/ronin-emblema.png` (emblema circular, PNG transparente, 512×512). Los íconos (`app/icon.png` 512×512, `app/apple-icon.png` 180×180 sin transparencia, `app/favicon.ico` 16/32/48) se regeneran desde el emblema.

Ajustar el fondo

`components/fondo-dojo.tsx`: opacidad y posición del ronin (`opacity-[0.08]`), halo cian, patrón de circuito, niebla y grano. Mantenerlo tenue para no competir con el contenido.

Cambiar textos

Los textos están en español dentro de cada página/componente:

- Alumno: `app/votar/[slug]/page.tsx` y `votacion.tsx` (incluye los 3 pasos de "Cómo funciona").
- Mentor: `app/mentor/**`.
- Etiquetas de tipo de sesión: `lib/temas.ts`.

Integración con GHL (fase 5)

- **Como enlace o botón:** usar `https://classvote-lac.vercel.app/votar/<cinturón>` (también se copia desde el panel con **Copiar enlace**).
- **Incrustado (iframe):** probar en Safari/iPhone. Si el navegador bloquea `localStorage` dentro del iframe, `lib/fingerprint.ts` genera un identificador nuevo en cada carga y el alumno podría votar de nuevo al recargar. Alternativas: abrir en pestaña nueva, o identificar al alumno con un parámetro firmado desde GHL.

Pruebas locales de la base de datos

Las migraciones se probaron en un Postgres local simulando Supabase:

```
-- "shim" mínimo antes de correr las migraciones
create role anon nologin; create role authenticated nologin;
create schema auth;
create table auth.users (id uuid primary key, email text, raw_user_meta_data jsonb);
create function auth.uid() returns uuid language sql stable as
  $$ select nullif(current_setting('request.jwt.claim.sub', true), '')::uuid $$;
grant usage on schema public, auth to anon, authenticated;
grant execute on function auth.uid() to anon, authenticated;
alter default privileges in schema public grant all on tables to anon, authenticated;
alter default privileges in schema public grant all on functions to anon, authenticated;
create publication supabase_realtime;
```

Luego: `0001` → `seed` → `0002` → `0003` → `0004` (sin la sección de `pg_cron`). Para actuar como un usuario: `set role authenticated; set request.jwt.claim.sub = '<uuid>';`.

Revisar el diseño sin datos reales

Crear temporalmente una página (por ejemplo `app/zz-preview/page.tsx`) que renderice los componentes con datos de ejemplo, correr con variables falsas (`NEXT_PUBLIC_SUPABASE_URL=http://127.0.0.1:9` `NEXT_PUBLIC_SUPABASE_ANON_KEY=x`), tomar capturas y **borrarla antes de subir**. Así se detectó, por ejemplo, que llamar desde el servidor a una función de un archivo `"use client"` rompía el prerender.

7 · Historial

Registro de lo que se construyó, en orden. Los detalles técnicos de cada paso están en los Pull Requests de GitHub.

Plan de fases

Fase	Alcance	Estado
1	Esqueleto Next.js + esquema Supabase + Vercel conectado	✓
2	Vista del alumno: proponer, votar, conteo en tiempo real	✓
3	Panel del mentor: cuentas, aprobación, gestión del cinturón, administración	✓
4	Cierre automático del ciclo a la hora de la clase	✓
—	Rediseño visual con la identidad RONIN	✓
5	Publicar cada <code>/votar/<cinturón></code> en su grupo de GHL	⌚ Tras la revisión y aprobación del equipo

Fase 1 · Esqueleto (23 sep 2026)

- Proyecto Next.js 16 + TypeScript + Tailwind v4 con clientes de Supabase para navegador y servidor.
- Esquema inicial (`0001_init.sql`) y los 7 cinturones (`seed.sql`), aplicados en Supabase.
- El repositorio se creó en `SantiJimenezIA/classvote` y luego se **transfirió a `dojoteamec-ship-it/classvote`**, porque Vercel (plan Hobby) solo conecta repos de la cuenta de GitHub con la que se inició sesión.
- Proyecto `classvote` en Vercel conectado al repo, con las 3 variables de entorno (la service role key se regeneró tras haberse expuesto en un chat).
- Rama de producción: `main`.

Fase 2 · Vista del alumno — PR #1

- `/votar/[slug]` con la próxima clase, proponer tema (nombre opcional) y votar varios temas, una vez cada uno.
- `0002`: conteo `votos_count` por trigger (el navegador nunca lee `votos`), permisos por columna para `anon`, `ciclo_actual()` (el ciclo se abre al entrar) y Realtime.
- Decisiones: varios temas con un voto cada uno, nombre opcional, ciclo que se abre al entrar.

Fase 3 · Mentores y administración — PR #1

- Registro propio del mentor; queda **pendiente** hasta que un admin lo aprueba.
- Panel por cinturón: temas en vivo (incluidos los ocultos), tipo de sesión, cerrar/reabrir, ocultar/mostrar.
- `/admin` : aprobar, rechazar, desactivar, asignar cinturones y rol admin.
- `0003` : estado y rol del mentor, trigger de registro, `es_admin()` y `puede_gestionar()` , RLS del panel, temas ocultos.
- Decisiones: aprobación del admin (en vez de código de invitación) y confirmación de correo desactivada.
- Configuración de Supabase corregida en el camino: proveedor Email encendido, Confirm email apagado, Site URL de producción.

Ajustes + Fase 4 · Cierre automático — PR #2

- Error real de Supabase visible al registrarse; el formulario conserva los datos si falla.
- `/admin` → **Restablecer contraseña** (temporal) y `/mentor/cuenta` para cambiarla.
- Mensaje claro cuando el correo no está confirmado.
- `0004` : `cerrar_ciclos_vencidos()` + `pg_cron` cada 5 minutos; `reabierto` para que una votación reabierto por el mentor no se vuelva a cerrar sola.
- Decisión: cierre a la hora de inicio de la clase.

Rediseño visual — PR #3 y PR #4

- **v1 (PR #3):** estética "dojo nocturno" (índigo, dorado, samurái dibujado en SVG). Se consideró demasiado básica y ajena a la marca.
- **v2 (PR #4):** estructura tipo Apple con la identidad real RONIN (ronin cibernético):
 - Logotipo RONIN y emblema del ronin tomados del logo oficial; favicon e íconos de app generados desde el emblema.
 - Negro azulado con **cian eléctrico** como único acento; tipografía Inter grande.
 - Alumno en dos columnas: título, **cuenta regresiva** al cierre y "Cómo funciona" / proponer y ranking; sello 印 cian al votar.
 - Cuenta regresiva también en el panel del mentor.
- Durante el rediseño el repo pasó a privado y Vercel bloqueó el despliegue (plan Hobby con merge hecho por otra cuenta). Se volvió a **público** y se publicó con **Promote** de un Preview.

Documentación — PR #5

- `README.md` y `docs/01` a `docs/07` (este paquete).

Pendientes y próximos pasos sugeridos

- **Fase 5:** publicar los enlaces en GHL tras la aprobación del equipo. Si se incrustan en iframe, probar en Safari/iPhone.
- **Recuperación de contraseña por correo:** conectar un SMTP propio (por ejemplo, Postmark) en Supabase → Authentication → SMTP.
- **Tipos generados:** reemplazar `types/database.ts` por los generados con el CLI de Supabase.
- **Posibles mejoras:** límite de votos por alumno, reordenamiento animado del ranking, historial de clases pasadas para el admin, exportar temas de una clase.